# Credit Card Approval Prediction

## (Automated Screening Using Machine Learning)

### Project Description

Banks and financial institutions receive thousands of credit card applications every day. A significant portion of these are rejected due to factors such as high existing loan balances, insufficient income levels, or excessive credit inquiries. Manually reviewing each application is both time-consuming and highly prone to human error, making it an inefficient process at scale.

This project automates the credit card approval decision using machine learning. By training a predictive model on historical applicant data, the system evaluates financial and demographic inputs to determine whether an applicant is likely to be approved or rejected, just as real banks do. Four classification algorithms are applied: Logistic Regression, Random Forest, XGBoost (Gradient Boosting), and Decision Tree.

The best-performing model is saved and integrated into a Flask web application. The project also includes an IBM Watson Machine Learning deployment pipeline, enabling the solution to be hosted on the cloud for scalable, real-time credit card approval predictions accessible through an intuitive user interface.

### Problem Statement

Manual review of credit card applications is slow, inconsistent, and highly prone to human error, particularly when analysts must weigh multiple factors such as existing loan balances, income levels, and the volume of recent credit inquiries. This creates a bottleneck when processing thousands of applications daily and increases the risk of inconsistent approval decisions. There is a need for a data-driven, automated system that can accurately classify credit card applications as approved or rejected based on applicant financial and demographic attributes, improving decision speed, consistency, and reliability for financial institutions.

### Scenarios

> **Scenario 1: Automated Credit Card Application Screening**
>
> A bank credit analyst enters a new applicant's financial profile — including income type, employment duration, and credit history — into the web application. The model returns an instant approval or rejection prediction, enabling the analyst to prioritize applications that require further review.

> **Scenario 2: High-Risk Applicant Identification and Compliance Review**
>
> A financial compliance officer uses the platform to batch-screen applicants with past-due loan records. The feature engineering pipeline converts multi-class payment status codes into binary labels, allowing the model to clearly classify high-risk applicants as ineligible for a new credit card.

> **Scenario 3: Customer Self-Service Credit Card Eligibility Check**
>
> A prospective customer uses the web application to enter personal and financial details — such as income level, employment status, and credit history — before submitting a formal credit card application. The system instantly predicts the likelihood of approval, helping the customer understand their eligibility and reducing unnecessary application submissions.

### Objectives

- To collect and preprocess historical applicant data — including income levels, loan balances, and past payment status — for use in machine learning models.
- To engineer meaningful features, including converting multi-class payment status codes into binary "high risk" / "low risk" labels, and normalizing/scaling financial features for better model convergence.
- To train and compare four classification algorithms — Logistic Regression, Random Forest, XGBoost, and Decision Tree — using Scikit-learn, optimized through hyperparameter tuning (Grid Search / Random Search).
- To identify the best-performing model and save it for integration into a real-time prediction service.

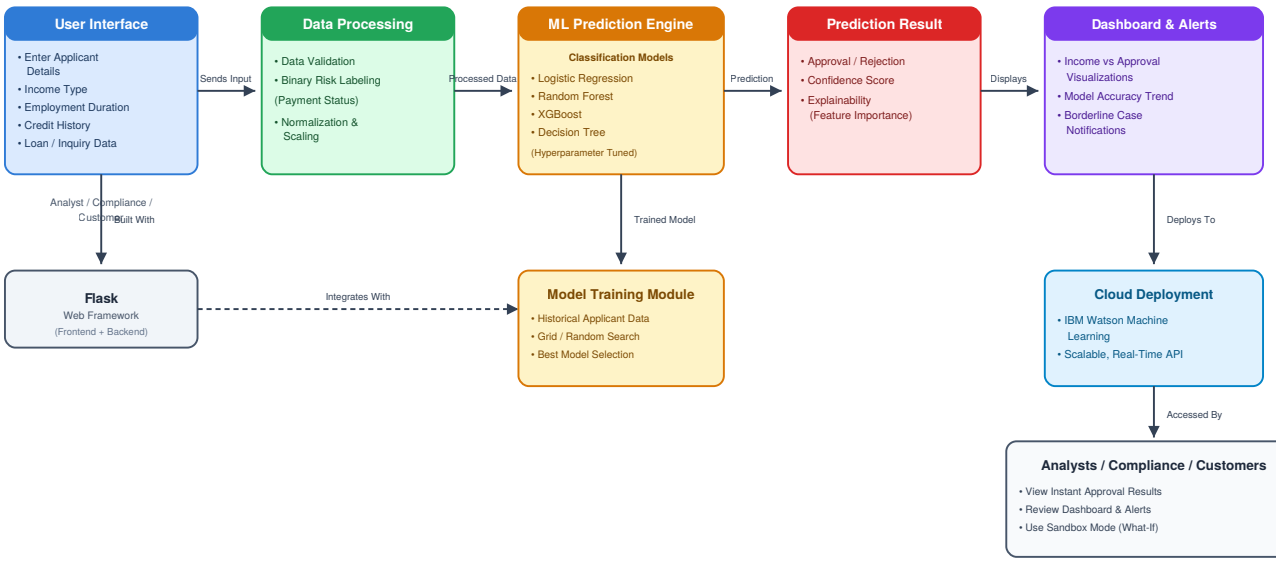
- To build a user-friendly Flask web application where bank analysts, compliance officers, and customers can input financial data and view instant approval/rejection predictions.
- To deploy the best-performing model on IBM Watson Machine Learning, enabling scalable, real-time predictions accessible on the cloud.

## Scope

The scope of this project is limited to binary classification of credit card applications (Approved / Rejected) based on applicant financial and demographic attributes such as income type, employment duration, credit history, and payment status. The solution is designed to serve three categories of users — bank credit analysts, compliance officers, and prospective customers — through a single Flask-based web interface, with the best-performing model deployed via IBM Watson Machine Learning for scalable, real-time cloud predictions.

## Technical Architecture

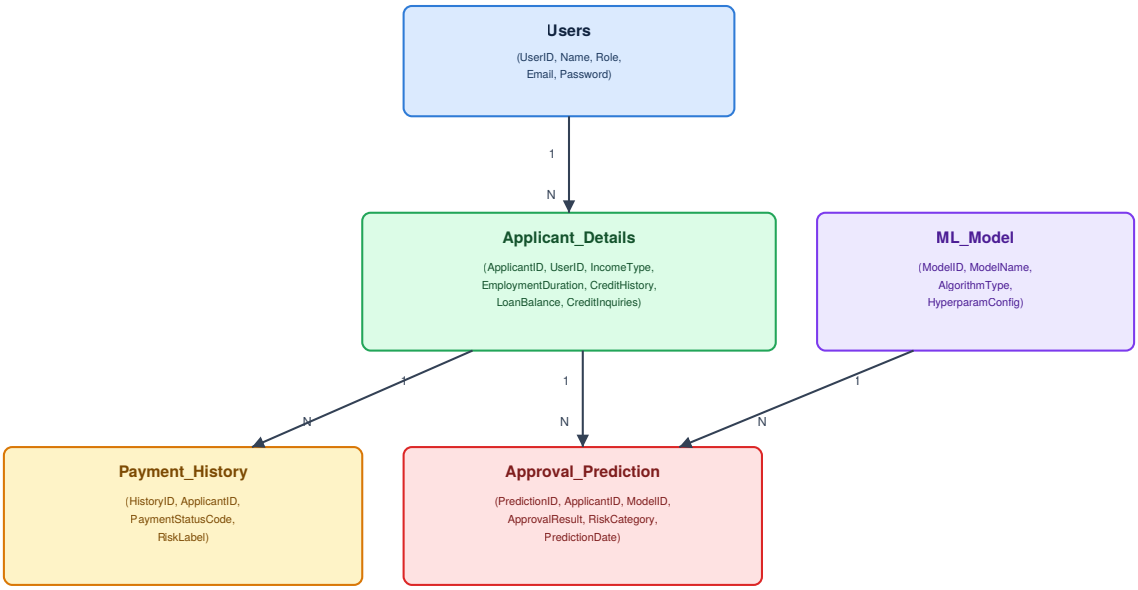
### Architecture Flow



## ER Diagram

### Description

The **Credit Card Approval Prediction System** follows a modular machine learning architecture built for fast, accurate approval decisions. Users — bank analysts, compliance officers, or customers — submit an applicant's financial profile through a **Flask-based web application**, where the data is validated and passed through the feature engineering pipeline (binary risk labeling, normalization, and scaling) before reaching the **Machine Learning Prediction Engine**. The system evaluates the application using trained **Logistic Regression, Random Forest, XGBoost, and Decision Tree** models, with the best-performing model selected for real-time predictions. The result (Approved or Rejected) is displayed instantly and can be reviewed later through the platform's dashboard. This architecture enables financial institutions to automate credit card approval, reduce manual effort, and deliver a consistent, scalable decision-making experience via IBM Watson Machine Learning.




## Pre-requisites

- Python Programming Language
- Machine Learning Algorithms — Logistic Regression, Decision Tree Learning, Random Forest, XGBoost
- Artificial Intelligence Fundamentals
- Data Handling & Numerical Computing — NumPy
- Model Development — Scikit-Learn
- Data Visualization — Matplotlib
- Feature Engineering & Data Preprocessing Techniques
- Web Framework — Flask
- Cloud Deployment — IBM Watson Machine Learning

## Project Workflow

**Milestone 1: Data Collection and Understanding**

- Activity 1.1: Collect historical applicant data, including income levels, loan balances, and past payment status.
- Activity 1.2: Analyze financial and demographic attributes — such as income type, employment duration, and credit history — relevant to the approval decision.

## Milestone 2: Feature Engineering and Preprocessing

- Activity 2.1: Convert multi-class payment status codes into binary "high risk" / "low risk" labels.
- Activity 2.2: Normalize and scale financial features to improve model convergence.

## Milestone 3: Model Building

- Activity 3.1: Train a Logistic Regression model as a baseline classifier.
- Activity 3.2: Train a Decision Tree classifier.
- Activity 3.3: Train a Random Forest ensemble model.
- Activity 3.4: Train an XGBoost (Gradient Boosting) model.
- Activity 3.5: Optimize each model using hyperparameter tuning (Grid Search / Random Search).

## Milestone 4: Model Evaluation and Selection

- Activity 4.1: Compare the four trained models and identify the best-performing algorithm.
- Activity 4.2: Save the best-performing model for integration into the web application.

## Milestone 5: Web Application & Cloud Deployment

- Activity 5.1: Build a Flask-based web application allowing analysts, compliance officers, and customers to input financial data.
- Activity 5.2: Integrate the trained model to generate instant approval/rejection predictions.
- Activity 5.3: Add a dashboard with visualizations, such as income vs. approval rates and model accuracy over time.
- Activity 5.4: Deploy the best-performing model using IBM Watson Machine Learning for scalable, real-time cloud predictions.

## Milestone 6: Enhancements

- Activity 6.1: Incorporate model explainability using SHAP values or feature importance plots so users understand which factors drive each decision.
- Activity 6.2: Add a notification system that alerts analysts of borderline cases requiring manual review.
- Activity 6.3: Provide a sandbox mode so customers can test different financial scenarios before formally applying.

## Methodology

### Dataset Description

| Attribute | Details |
|---|---|
| Dataset Type | Historical applicant financial & demographic records |
| Target Variable | Credit Card Approval Status (Approved / Rejected) |
| Feature Types | Categorical & Numerical — income type, employment duration, credit history, payment status, loan balances, credit inquiries |

### Data Preprocessing Steps

- **Multi-Class to Binary Conversion:** past-due payment status codes are converted into binary "high risk" / "low risk" labels, allowing the model to clearly classify high-risk applicants.
- **Feature Scaling:** financial features are normalized and scaled to improve model convergence, particularly for scale-sensitive algorithms such as Logistic Regression.
- **Feature Selection:** financial and demographic inputs — income type, employment duration, and credit history — are used as the core predictive features.
- **Hyperparameter Tuning:** each of the four models is optimized using Grid Search or Random Search to maximize predictive performance.

### Algorithms Used

- Logistic Regression
- Decision Tree
- Random Forest
- XGBoost (Gradient Boosting)

### Evaluation Metrics

- **Accuracy** — overall proportion of correctly classified applications.
- **Precision** — proportion of predicted approvals that were actually correct.
- **Recall** — proportion of actual approvals that were correctly identified.
- **F1-Score** — harmonic mean of Precision and Recall, useful for imbalanced classes.
- **Confusion Matrix** — detailed breakdown of true/false positives and negatives.
- **Approval / High-Risk Probability** — the model's confidence score behind each decision, surfaced to the user alongside the prediction.

### Model Comparison & Selection

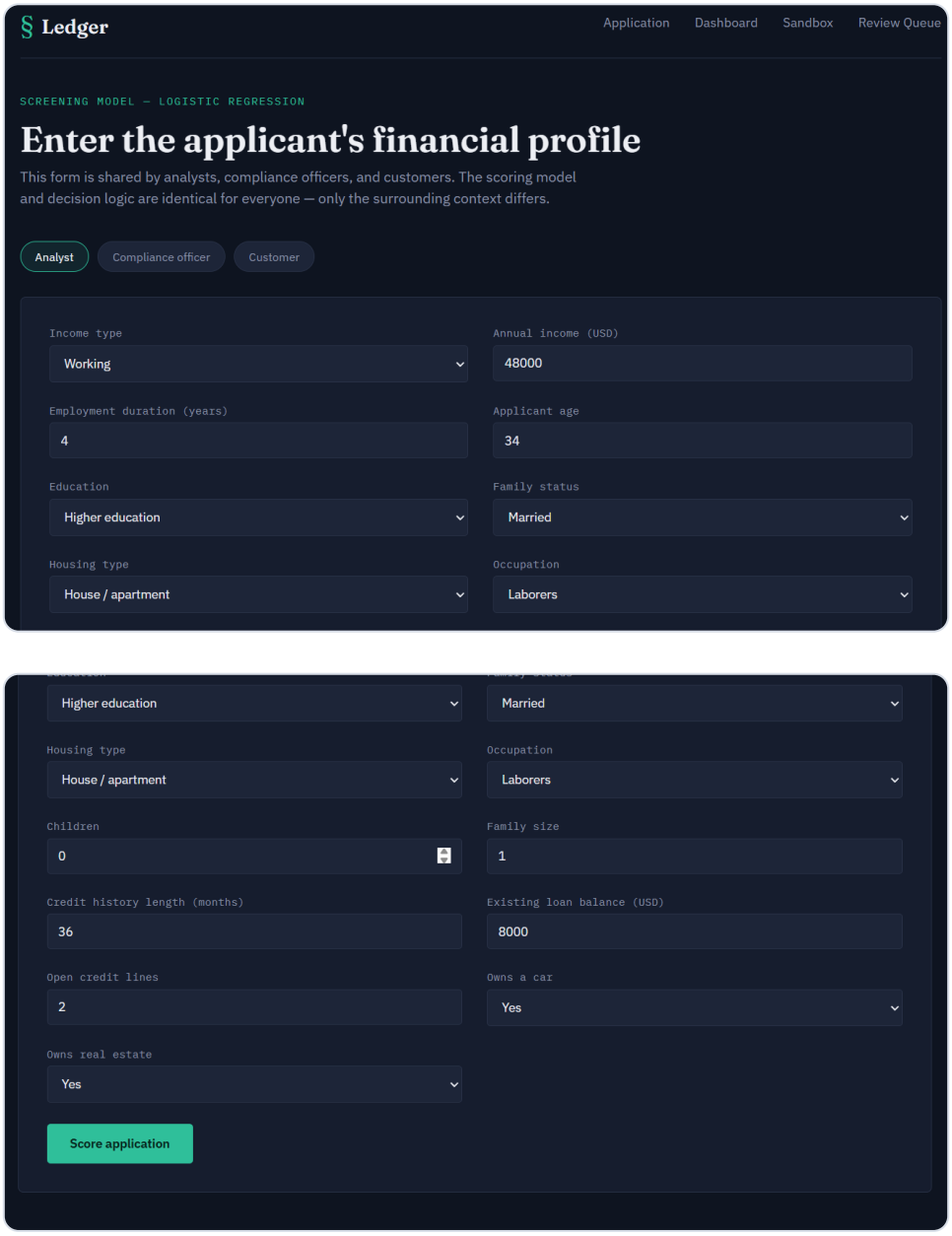
All four classification models are trained on the same historical applicant dataset and compared against one another using the metrics above. The best-performing algorithm is selected and saved for integration into the Flask web application, where it powers real-time approval and rejection predictions.

## Web Application

The best-performing model is deployed as an interactive web application that allows bank analysts, compliance officers, and customers to enter an applicant's financial profile — income type, employment duration, credit history, loan balance, and credit inquiries — and receive an instant **Approved / Rejected** prediction. The application is built with Flask and is deployed to the cloud via IBM Watson Machine Learning for scalable, real-time access.

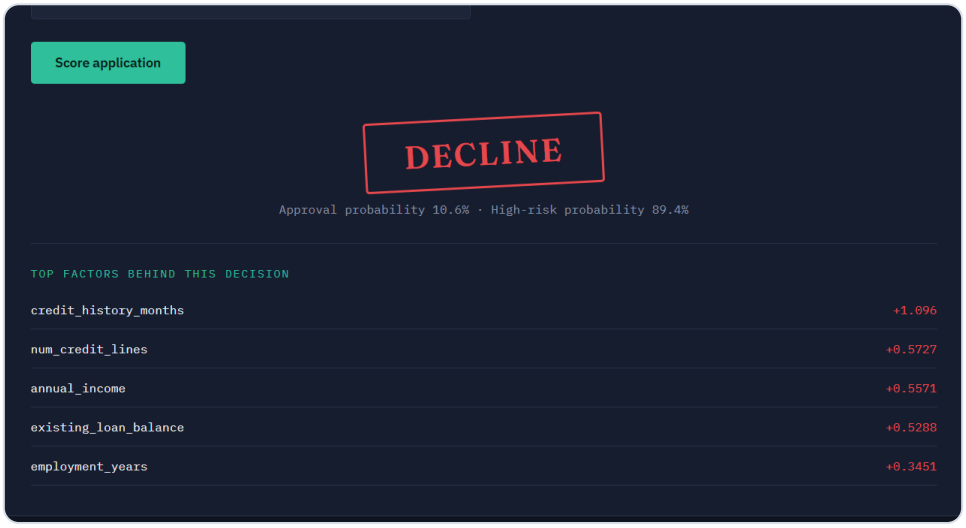
### Application Form

The form is shared across all three user roles — analyst, compliance officer, and customer — with a role selector at the top. The scoring model and decision logic remain identical for every role; only the surrounding context changes.




## Prediction of Result

Once submitted, the applicant profile is scored instantly. The result is displayed as a clear Approve/Decline stamp along with the approval probability, high-risk probability, and the top factors that drove the decision — giving the analyst, compliance officer, or customer immediate insight into the model's reasoning.


Score application
DECLINE
Approval probability 10.6% · High-risk probability 89.4%
TOP FACTORS BEHIND THIS DECISION
credit_history_months +1.096
num_credit_lines +0.5727
annual_income +0.5571
existing_loan_balance +0.5288
employment_years +0.3451

## Frontend Implementation

- HTML — structures the applicant data entry form and results view.
- CSS — styles the interface for a clean, professional look consistent across analyst, compliance, and customer views.
- JavaScript — handles form submission and displays the instant approval/rejection response returned by the backend.

## Backend Implementation Flask(app.py)

The Flask backend receives the applicant's financial profile from the web form, applies the same feature engineering pipeline used during training (binary risk labeling, normalization, and scaling), and passes the processed input to the saved best-performing model to generate a real-time prediction. The result — Approved or Rejected, with an approval and high-risk probability — is returned to the interface instantly.

### Cloud Deployment (IBM Watson Machine Learning)

The trained model is deployed through an IBM Watson Machine Learning pipeline, enabling the application to be hosted on the cloud. This ensures the credit card approval service is scalable and can serve real-time predictions on demand to bank analysts, compliance officers, and customers alike.

## Conclusion and Future Scope

### Conclusion

The Credit Card Approval Prediction project successfully demonstrates how machine learning can automate and improve the credit card screening process. By training and comparing Logistic Regression, Random Forest, XGBoost, and Decision Tree models on historical applicant data, the system accurately predicts whether an application is likely to be approved or rejected. The best-performing model is integrated into a Flask web application and deployed via IBM Watson Machine Learning, giving bank analysts, compliance officers, and customers instant, consistent, and scalable access to credit card approval predictions.

### Future Scope

- Incorporating model explainability — such as SHAP values or feature importance plots — so users understand which factors drive each approval decision.
- Adding a notification system that alerts analysts of borderline cases requiring manual review.
- Providing a sandbox mode so customers can test different financial scenarios before formally applying.
- Continuing to leverage IBM Watson Machine Learning for scalable, real-time predictions as application volume grows.